Mastishk: An Open Protocol for Structured, Persistent, Filesystem-Native Memory in AI Coding Agents

Karan Sharma
github.com/mastishk-ai/mastishk
mastishk.dev

May 2026

Abstract

We present Mastishk, an open protocol and reference implementation for persistent, structured memory in AI coding agents. Current AI coding agents—Claude Code, Cursor, GitHub Copilot, and GPT-based tools—are stateless by default. Each session begins with no knowledge of prior decisions, architecture choices, errors, or goals. Existing workarounds (context files, vector retrieval, proprietary memory layers) fail to scale, lack causal structure, and do not support multi-agent collaboration. Mastishk defines a three-layer memory architecture stored as markdown files with YAML frontmatter, organised as a directed acyclic graph, and versioned through git. A `mastishk inject` operation traverses the dependency graph from task-matched nodes rather than relying on vector similarity, preserving causal relationships. The protocol additionally specifies: (1) an `instruct` interface that generates agent-specific configuration for Claude Code, Cursor, Windsurf, and API agents in a single command; (2) automated ambient capture via post-commit git hooks; and (3) a `CLAUDE.md` integration format that ensures coding agents receive the correct memory workflow without manual configuration. Any implementation that passes the compliance suite is Mastishk-compliant, enabling interoperability across agents, tools, and programming languages. We release the protocol specification, reference TypeScript implementation, and compliance test suite as prior art under Apache 2.0.

1 Introduction

AI coding agents have become primary collaborators in software development. A developer working with Claude Code, Cursor, or a GPT-based assistant delegates significant cognitive work to the agent: code generation, architecture decisions, debugging, and refactoring. The agents are capable. The infrastructure around them is not.

Every session starts cold. The agent has no memory of the codebase decisions made last Tuesday, the architecture constraint added last month, or the three bugs that were logged and resolved last week. Developers compensate by maintaining context files (`CLAUDE.md`, `RULES.md`, system prompts), but these files go stale, hit token limits, and do not scale with project complexity.

Three categories of workaround exist, and all fail in predictable ways:

Flat context injection. A manually maintained text file is prepended to every session. This degrades attention as the file grows [3], goes stale because no agent reliably updates it, and hits token limits on large projects.

Vector RAG retrieval. Memory is embedded and retrieved by semantic similarity. This optimises for proximity in embedding space rather than causal relevance. A goal node and a dependent task may have low cosine similarity but a critical dependency relationship that RAG cannot reconstruct [2].

Proprietary memory layers. Services such as Mem0 [4], MemGPT/Letta [1], and LangMem store memory in vendor-specific infrastructure. Memory is inaccessible to agents from other vendors, not human-inspectable, and cannot be versioned with the project.

We propose Mastishk as a solution to all three failure modes. Mastishk is:

- **Filesystem-native** — memory is stored as markdown files, versioned by git, readable by humans and all agents without special tooling.
- **Causally structured** — context injection traverses a dependency graph, not an embedding index.
- **Multi-agent by design** — the memory folder belongs to the project; any agent reads and writes the same folder.
- **Protocol-first** — Mastishk defines a specification and compliance suite. Any implementation in any language that passes the compliance tests is Mastishk-compliant.

This paper describes the protocol design (Section 2), the six methodology claims (Section 3), the reference implementation (Section 4), a comparison with prior work (Section 5), and directions for empirical validation (Section 6).

2 Protocol Design

2.1 Memory structure

Every project using Mastishk contains a `.mastishk/` folder:

```
.mastishk/
  intent/          <- goals and constraints
  state/           <- tasks, progress, blockers
  knowledge/       <- decisions, architecture, stack
  issues/          <- errors, bugs, blockers
  agents.json      <- registered agents
  memory-map.json  <- DAG index
```

2.2 The node format

Every unit of memory is a `.msk.md` file — markdown with YAML frontmatter. A node looks like:

```
---
id: goal-launch-v1
type: goal
layer: intent
status: active
confidence: 1.0
created_by: human
```

```
created_at: 2026-05-12T00:00:00Z
last_updated_by: claude-code
last_updated_at: 2026-05-12T10:30:00Z
dependencies: []
contradictions: []
tags: [launch, npm, v1]
---

Ship @mastishk/cli to npm before June 2026.
Target: 100 GitHub stars within 30 days.
```

Required fields include: `id` (kebab-case, unique), `type` (goal / task / decision / constraint / issue / fact), `layer`, `status`, `confidence` (0.0–1.0), creator fields, timestamps, `dependencies`, and `contradictions`. Optional fields include `tags`, `scanned`, `verified`, and `expires_at`.

2.3 Layer semantics

The three primary layers have distinct update rules:

`intent/` Long-term goals and constraints. Changes rarely. Agents must not modify without human confirmation.

`state/` Current tasks, progress, blockers. Changes every session. Writes are *append-only*: new content is appended with a timestamp header, existing content is never overwritten.

`knowledge/` Architecture decisions, technical facts. Grows over time. Writes replace the body content but preserve frontmatter.

The `issues/` layer is a dedicated log: every issue is append-only (never deleted), only resolved via `status: resolved`.

2.4 The memory graph

`memory-map.json` is the DAG index of all nodes:

```
{
  "version": "0.1",
  "updated_at": "2026-05-12T10:30:00Z",
  "nodes": {
    "goal-launch-v1": {
      "file": "intent/goal-launch-v1.msk.md",
      "layer": "intent", "type": "goal",
      "status": "active", "confidence": 1.0,
      "dependencies": [], "tags": ["launch"]
    },
    "task-write-spec": {
      "file": "state/task-write-spec.msk.md",
      "layer": "state", "type": "task",
      "status": "active", "confidence": 0.8,
      "dependencies": ["goal-launch-v1"]
    }
  },
  "edges": [
    { "from": "task-write-spec", "to": "goal-launch-v1",
```

```
      "relation": "depends-on" }
  ]
}
```

A compliant implementation must enforce: (1) no cycles, (2) no dangling references, (3) unique IDs. Violations raise typed errors.

2.5 The inject algorithm

`inject(task)` is the primary interface between Mastishk and an agent. It accepts a natural language task description and returns a formatted context string:

1. Tokenise the task string and match against node IDs, tags, and body content
2. Select seed nodes from matches, ranked by confidence descending
3. Traverse the graph from seeds at depth 2
4. Filter to `status: active` nodes with `confidence` ≥ 0.5
5. Sort: `intent` first, then `state`, then `knowledge`
6. Format output per section 6.3 of the spec
7. Enforce a 2000-token budget, dropping lowest-confidence nodes first

Output format:

```
## Mastishk context -- build JWT authentication
Generated: 2026-05-12T10:30:00Z

### Goals
**goal-launch-v1** [1.0] -- Ship @mastishk/cli to npm before June 2026.

### Current state
**task-write-spec** [0.8] -- Write the protocol spec.

### Knowledge
**fact-stack-choice** [0.9] -- Using TypeScript with strict mode.
Full stack: TypeScript 5.4, Node 20, Zod 3.23.

### Open issues
```

A key property: if a node has `confidence` ≥ 0.9, its full body is included. Otherwise, only the first sentence is shown. This incentivises high-confidence, concise node bodies for token efficiency.

2.6 The write-back contract

Any agent writing to Mastishk memory must:

1. Provide its agent ID in `last_updated_by`
2. Update `last_updated_at` to the current timestamp
3. Pass the new node through the Referee before committing

The Referee checks incoming writes against the `contradictions` field of existing nodes. If a contradiction is detected, the write is rejected and a conflict issue node is created in `issues/` with tags `[conflict, referee]`.

3 The Six Methodology Claims

The following six ideas constitute the Mastishk methodology, claimed as prior art as of publication of this paper.

Claim 1: Three-layer semantic memory. Agent memory is divided into three semantically distinct layers with different persistence, update frequency, and trust levels. This separation mirrors how human teams organise project knowledge and maps naturally to how agents should weight different types of memory.

Claim 2: DAG traversal for context injection. Relevant memory is identified by traversing a directed acyclic graph from dependency-linked seed nodes, not by vector similarity or recency. This preserves causal structure: if Task A depends on Goal B constrained by Constraint C, all three are returned together.

Claim 3: Filesystem-native markdown nodes. Every memory unit is a `.msk.md` file with YAML frontmatter. Human-readable, git-versionable, portable, requiring no database or binary format.

Claim 4: Git as episodic memory. The git history of `.mastishk/` is the agent's episodic memory. `mastishk checkpoint` commits the current state; the git log is queryable as a timeline of agent cognition.

Claim 5: Confidence-weighted nodes with Referee. Every node carries a confidence score (0.0–1.0). A Referee layer checks all incoming writes against existing contradictions. Silent overwrites of conflicting beliefs are impossible.

Claim 6: Ambient memory capture. Memory stays current through post-commit git hooks and agent MCP tool use, without developer overhead. The `mastishk init` command installs a post-commit hook that classifies commits by prefix (`feat:`, `fix:`, `wip:`) and appends structured entries to the state layer automatically. The `mastishk instruct --agent <name>` command generates agent-specific configuration so any supported coding agent (Claude Code, Cursor, Windsurf, or a custom API agent) is immediately briefed on project memory.

4 Reference Implementation

The reference implementation is written in TypeScript (strict mode) and published as three npm packages:

- `@mastishk/core` (v0.1.0) — schema (Zod), validation, read (`gray-matter`), write (append/replace logic), graph (DAG operations), traversal, inject.

- `@mastishk/cli` (v0.1.0) — `mastishk` CLI with commands: `init`, `inject`, `update`, `log`, `scan`, `status`, `checkpoint`, `instruct`, `hook`.
- `@mastishk/mcp` (v0.1.0) — MCP server exposing five tools: `mastishk_inject`, `mastishk_update`, `mastishk_log`, `mastishk_status`, `mastishk_checkpoint`.

4.1 The instruct pipeline

`mastishk instruct --agent <name>` is a one-command agent onboarding interface. For `claude-code`, it writes or updates the `CLAUDE.md` workflow briefing (spec section 13). For `cursor` and `windsurf`, it writes the appropriate MCP config file (`.cursor/mcp.json` or `.windsurf/mcp.json`). For `api`, it prints the full `inject` output to stdout for piping into any custom agent. After the agent-specific action, it prints a memory summary showing current goals, status, open issue count, and last-updated agent.

4.2 Git hook automation

`mastishk init` installs a post-commit hook at `.git/hooks/post-commit`. On every commit, `mastishk hook` reads the commit message, classifies it by prefix (`feat:`/`add:`, `fix:`, `wip:`), and appends a structured entry to the primary state task node. This implements claim 6 (ambient capture) without any developer action beyond conventional commit message prefixes.

4.3 CLAUDE.md integration

`mastishk init` writes a structured workflow briefing to `CLAUDE.md` in the project root. If the file exists, the section is appended; if it is absent, the file is created. The briefing instructs Claude Code to run `mastishk inject` at session start, update state during work, and run `mastishk checkpoint` at session end. This closes the loop between human commits, agent sessions, and memory persistence without additional tooling.

4.4 Key implementation decisions

gray-matter for frontmatter parsing. The `gray-matter` library parses YAML frontmatter, but `js-yaml` (its YAML engine) automatically coerces ISO 8601 strings to JavaScript `Date` objects. We normalise date fields back to ISO strings immediately after parsing to satisfy Zod's string validation.

Append-only state writes. State layer writes prepend a timestamp header (`### {ISO} --- {agent}`) before new content and append to the existing body. This means state nodes accumulate a timestamped log of all progress updates across sessions.

updateNodeMeta. A metadata-only update path is required for the CLI `update` command (changing `status`/`confidence` without triggering state-layer append). This is a SPEC-GAP in the current protocol version.

Token budget. The spec defines a 2000-token budget but does not specify tokenisation. We approximate 1 token = 4 characters (CHARS_PER_TOKEN = 4). Nodes are dropped in ascending confidence order until the budget is satisfied.

4.5 Compliance suite

The compliance suite at `tests/compliance/` contains 163 tests across 7 files, one per spec section. It is designed to be language-agnostic: a future Python or Go implementation that passes all tests is Mastishk-compliant. The full test suite (including unit tests for the CLI, MCP, and new `instruct`/`hook` commands) totals 380 tests across 18 test files.

File	Spec section	Tests
`01-node-format`	§3 Node format	36
`02-layer-semantics`	§4 Layer semantics	14
`03-graph-constraints`	§5 Graph	26
`04-inject-format`	§6 Inject	15
`05-write-back`	§7 Write-back	11
`06-mcp-tools`	§9 MCP tools	29
`07-cli-commands`	§10 CLI	32

Table 1: Compliance suite coverage by spec section

5 Related Work

System	Storage	Structure	Multi-agent	Open protocol
Mem0 [4]	Hosted vector DB	Flat	No	No
Graphiti [5]	Neo4j	Graph	No	No
MemGPT/Letta [1]	In-process	Hierarchical	No	No
LangMem	Vector + SQL	Flat	No	No
agentmemory	Markdown	Flat	No	No
Mastishk	**Filesystem**	**DAG (3-layer)**	**Yes**	**Yes**

Table 2: Comparison of agent memory systems

The most relevant prior work is MemGPT [1], which introduces hierarchical in-context and out-of-context memory for LLM agents. Mastishk differs in three ways: (1) it targets coding agents specifically, (2) it uses the filesystem as the persistence layer rather than an in-process store, and (3) it is a multi-agent protocol rather than a single-agent library.

GraphRAG [2] and related work on knowledge graphs for retrieval shares the insight that graph structure improves relevance over flat retrieval. Mastishk applies this insight specifically to project-level agent memory, with a three-layer semantic structure tailored to software development context.

6 Empirical Hypotheses

The following hypotheses are intended for future empirical validation:

H1 — Task continuity. An agent using Mastishk memory completes a multi-session coding task in fewer sessions and with fewer contradictions than an agent using flat context injection or no memory.

H2 — Context efficiency. `mastishk inject` delivers equivalent or better task performance at significantly lower token cost than full context injection, by traversing only the causally relevant subgraph.

H3 — Contradiction reduction. The Referee layer reduces the rate at which agents produce outputs contradicting prior decisions, compared to agents operating without structured memory.

These hypotheses are testable using the Claude API with reproducible task sets. Experimental results will be published at https://mastishk.dev.

7 Conclusion

We have presented Mastishk, an open protocol for structured, persistent, filesystem-native memory in AI coding agents. The core contribution is a three-layer DAG architecture with causal context injection, supported by a compliance suite that enables interoperability across implementations.

The reference implementation is available at https://github.com/mastishk-ai/mastishk and on npm as `@mastishk/core`, `@mastishk/cli`, and `@mastishk/mcp`. All described methodologies are released as prior art under Apache 2.0.

References

[1] Packer, C., Fang, V., Patil, S. G., Lin, K., Wooders, S., & Gonzalez, J. E. (2023). *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560.

[2] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., ... & Larson, J. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130.

[3] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. arXiv:2307.03172.

[4] Mem0.ai. (2024). *Mem0: The Memory Layer for AI Agents*. https://mem0.ai.

[5] Zep AI. (2024). *Graphiti: Temporally-aware Knowledge Graph for AI Agents*. https://github.com/getzep/graphiti.